ChatGPT 답변:

첨부해 주신 Lighthouse 13.5.0 리포트를 기준으로 분석하겠습니다.

우선 현재 측정 결과에서 가장 중요한 점은 **FCP와 LCP가 모두 2초 초반에 형성되어 있다는 것**입니다. 따라서 단순히 이미지 용량만 줄이는 방식보다는 초기 렌더링 경로와 리소스 로딩 순서를 함께 최적화하는 것이 좋겠습니다.

1. 현재 성능 측정 결과

FCP	LCP
2.12초	**2.27초**
목표: 1초대	목표: 1초대
Speed Index	LCP 점수
2.12초	**0.94**

측정 환경: 모바일 에뮬레이션 · Lighthouse 13.5.0

현재 상태는 나쁘지 않습니다. 다만 목표가 FCP와 LCP 모두 1초대라면 각각 약 120ms 이상, 270ms 이상의 단축이 필요합니다. 여유 있게 목표를 달성하려면 1.5~1.8초 수준을 노리는 편이 좋습니다.

2. 실제로 수정해야 할 부분

리포트의 세부 진단을 확인하면 우선순위가 꽤 명확합니다.

① CSS 렌더링 차단 해소 — 최우선

1순위

문제 파일

`assets/index-Di6fmtab.css`

항목	측정값
전송 크기	11.04KB
원본 크기	60.4KB
렌더링 차단 시간	456ms

Lighthouse의 렌더링 차단 요청 진단

이 부분은 가장 먼저 손대는 게 좋겠습니다.

CSS는 브라우저가 DOM을 그리기 전에 스타일 계산을 완료해야 하므로 초기 렌더링을 지연시킬 수 있습니다.

수정 방향

- 첫 화면에 필요한 CSS와 나머지 CSS 분리
- 초기 화면의 핵심 스타일만 Critical CSS로 인라인 처리
- 사용하지 않는 CSS 제거
- 페이지별 CSS 분리 및 지연 로딩 검토
- Tailwind를 사용한다면 불필요한 유틸리티 생성 여부 확인

단, CSS 전체를 무조건 비동기 로딩하면 화면 깜빡임(FOUC)이 생길 수 있습니다. 첫 화면에 필요한 스타일은 즉시 적용해야 합니다.

② Pretendard 외부 CDN 의존성 제거

2순위

현재 폰트 CSS를 외부 CDN에서 가져오고 있습니다.

```HTML
https://cdn.jsdelivr.net/gh/orioncactus/pretendard@v1.3.9/dist/web/variable/pretendardvariable.min.css
```

확인된 문제는 다음과 같습니다.

- 외부 CDN 연결 필요
- CSS 요청이 초기 로딩 경로에 추가됨
- 해당 CDN의 preconnect가 실제 요청과 제대로 연결되지 않음
- Lighthouse에서 미사용 연결 경고 발생

추천 수정

Pretendard 폰트를 프로젝트 내부에 저장하고 직접 제공하는 방식을 추천합니다.

CSS

```css
@font-face {
  font-family: "Pretendard";
  src: url("/fonts/PretendardVariable.woff2")
       format("woff2");
  font-weight: 100 900;
  font-display: swap;
}
```

그리고 HTML에서 외부 폰트 CSS를 제거합니다.

HTML

```html
<!-- 기존 외부 CDN 제거 -->
```

폰트 파일은 WOFF2를 사용하고, 실제 첫 화면에서 필요한 폰트만 preload하는 것이 좋습니다.

③ JavaScript 초기 로딩 최적화

3순위

현재 JS 리소스는 다음과 같습니다.

파일	원본 크기
index JS	109KB
React	274.7KB
Query	38.5KB
Utils	124.7KB
Icons	18.2KB
Runtime	0.7KB

압축 전 리소스 크기 기준

전체적으로 약 566KB입니다.

특히 React 번들의 실행 시간이 눈에 띕니다.

- React 번들 관련 실행 시간: 약 129ms
- 그중 JavaScript 실행: 약 120.6ms
- 초기 JS 부팅 시간: 약 128ms

또한 Lighthouse는 미사용 JavaScript 약 75.3KB를 확인했습니다.

수정 방향

1. 초기 화면에서 사용하지 않는 컴포넌트 `React.lazy()` 로 분리
2. 설정 모달, 상세 모달, 알림 관련 기능 지연 로딩
3. 페이지별 라우트 단위 코드 스플리팅
4. 사용하지 않는 라이브러리와 유틸리티 제거
5. 아이콘 라이브러리의 전체 import 여부 확인

예를 들어:

```typescript
const SettingsModal = lazy(
  () => import("./SettingsModal")
);

const InsuranceDetailModal = lazy(
  () => import("./InsuranceDetailModal")
);
```

다만 React 자체를 무작정 쪼개는 것보다는 실제 초기 화면에 필요한 코드와 그렇지 않은 코드를 구분하는 것이 중요합니다.

④ 초기 화면 렌더링 구조 개선

여기서 중요한 사실이 하나 있습니다.

Lighthouse가 검출한 현재 LCP 요소는 이미지가 아닙니다.

실제 LCP 요소

2026년 11월

```html
<h2 class="text-[20px] lg:text-[22px]
  font-extrabold text-gray-900 tracking-tight">
```

하위 항목	시간
첫 바이트까지의 시간	9ms
요소 렌더링 지연	132ms

LCP 요소 위치: 화면 상단에서 약 365px

이건 상당히 중요한 단서입니다.

현재 LCP가 이미지 다운로드에 발목 잡히는 상황은 아닙니다. 따라서 이미지 압축이나 이미지 preload에 집중하는 것은 우선순위가 낮습니다.

오히려 다음을 살펴봐야 합니다.

- 초기 화면에서 React가 실제 DOM을 구성하는 시점
- CSS 적용과 스타일 계산 시점
- 초기 화면의 컴포넌트 렌더링 구조
- 불필요한 초기 상태 계산 및 데이터 의존성

특히 해당 `h2`가 있는 달력 메인 화면을 중심으로 살펴보시는 게 좋겠습니다.

⑤ API 호출은 우선순위가 낮습니다

현재 측정된 API 요청을 보면 다음과 같습니다.

API	요청 시작	응답 완료
insurance-products	132ms	141ms
notifications	134ms	144ms

요청 자체는 빠릅니다.

따라서 지금은 API 응답 속도를 개선하기보다는 초기 화면 렌더링을 먼저 최적화하는 것이 합리적입니다.

단, 실제 운영 서버에서도 동일한 응답 속도가 나온다는 의미는 아닙니다.

---

3. 수정 우선순위 최종 정리

제가 개발한다면 다음 순서로 작업하겠습니다.

1. Critical CSS 최적화 최우선

- 초기 화면 CSS 인라인 처리 검토
- 메인 CSS에서 불필요한 규칙 제거
- CSS 로딩 경로 단축

---

2. Pretendard 로컬화 높음

- 외부 CDN 제거
- WOFF2 적용
- 필요한 폰트만 preload

3. JavaScript 코드 스플리팅 높음

- 모달 및 설정 컴포넌트 분리
- 초기 화면에 불필요한 기능 lazy loading
- 미사용 JS 75.3KB 제거 검토

4. 초기 화면 렌더링 최적화

- 달력 메인 컴포넌트 렌더링 구조 점검
- 초기 상태 계산 최소화
- 불필요한 컴포넌트 렌더링 방지

---

5. 재측정 및 검증

- 동일한 Lighthouse 모바일 환경
- FCP, LCP 비교
- 변경 전후 Performance Trace 비교

4. 목표 성능

현재 측정값과 목표를 비교하면 다음과 같습니다.

측정 항목	현재	목표
FCP	2.12초	1.5초 이하
LCP	2.27초	1.5초 이하
Speed Index	2.12초	1.5초 이하

참고로 CSS 차단 456ms나 JS 실행 120ms를 각각 제거한다고 해서 그만큼 FCP가 단축되는 것은 아닙니다. 서로 겹치는 구간이 있기 때문에 수정 후 실제 측정으로 확인해야 합니다.

5. 추가로 필요한 자료

현재 리포트만으로도 수정 우선순위는 잡을 수 있지만, 실제 코드 수정 위치까지 특정하려면 소스 코드가 필요합니다.

특히 다음 파일을 확인하고 싶습니다.

- `index.html`
- `vite.config.ts`
- `src/main.tsx`
- `src/App.tsx`
- 메인 달력 페이지 컴포넌트
- `index.css` 또는 전역 CSS
- `package.json`

이 파일들을 올려주시면 단순히 일반적인 최적화 방법을 알려드리는 게 아니라, **어떤 파일의 어떤 코드를 어떻게 변경해야 FCP/LCP를 줄일 수 있는지** 구체적으로 분석해 드리겠습니다.

붙여넣은 텍스트(1)(1).txt

붙여넣은 텍스트(1)(1).txt

붙여넣은 텍스트(1)(1).txt

붙여넣은 텍스트(1)(1).txt

붙여넣은 텍스트(1)(1).txt

개인적으로는 지금 단계에서 무작정 라이브러리를 제거하기보다는 CSS 렌더링 차단과 초기 JS 실행 경로부터 개선하는 것을 추천합니다.